

# Lab questions 202506

## Question 4 - 1x

### Objective

To assess students' understanding of how unquoted wildcards in cron-triggered scripts can be abused to achieve privilege escalation on Unix systems.

### – FROM HERE

---

### Scenario

A system administrator has set up a daily backup cronjob that runs as root. Every night at 02:00 the following command is executed from the directory /home/backup:

```
tar czf /var/backups/home-$(date +%F).tar.gz *
```

All files in /home/backup are owned by a normal user and world-writable.

### Questions

1. Explain how an attacker with write access to /home/backup could hijack this cronjob to execute arbitrary code as root.
  - a. List the exact filenames (including any special syntax) the attacker would create in /home/backup to exploit this vulnerability.
2. Describe the underlying mechanism in tar that causes the attacker's file(s) to be interpreted as command-line options rather than data.
  - a. What component/program is responsible for wildcard characters expansion?

b. What is the difference between a wildcard and a regular expression?

## – TO HERE

---

### Possible Answer

An attacker abuses the fact that the shell expands `*` without quotes, passing malicious filenames to tar as options. By creating two files in `/home/backup` named exactly:

```
–checkpoint=1
```

```
–checkpoint-action=exec=/home/backup/myshell.sh
```

and placing an executable script `myshell.sh` that opens a reverse shell, when `tar czf ... *` runs it becomes:

```
tar czf /var/backups/home-2025-06-11.tar.gz –checkpoint=1
```

```
–checkpoint-action=exec=/home/backup/myshell.sh myshell.sh
```

Because GNU tar supports the `--checkpoint-action=exec` feature, it will run `myshell.sh` as root at the first checkpoint. Thus the attacker gains a root shell.

Wild characters are expanded by the shell, that is why the above exploit works: tar has no clue where those parameters/filenames are coming from (directly from user or from wildcard expansion).

A shell wildcard is a simpler pattern-matching mechanism that uses special characters (like `*` and `?`) to represent other characters. Wildcards are expanded into filenames

(matching the wildcard) by the shell. While a regular expression is a more powerful and flexible pattern-matching tool with a more complex syntax, allowing for more sophisticated patterns to be matched.

## Question 5 - 1.5x

### Objective

To evaluate students' ability to design a return-to-libc exploit against a 64-bit Linux binary protected by NX but lacking PIE and stack canaries.

### – FROM HERE

---

### Scenario

You have a 64-bit Linux program compiled with the options

`-no-pie -z noexecstack -fno-stack-protector`

In its source, among others, you identified this function:

```
void greet() {  
    char buf[128];  
    printf("Name? ");  
    gets(buf);  
    printf("Hello, %s\n", buf);  
}
```

ASLR is off on this system.

The libc version in use is known, and you have a local copy.

The binary is setuid root.

## Questions

1. Outline a step-by-step ROP chain that allows you to **drop a shell with root UID**. Justify your answer.
2. What is the reason this kind of exploit requires stack alignment? How should the stack be aligned? Explain your answer.
3. [for extra points] Can you make the program exit gracefully with your ROP chain? Explain how.

## – TO HERE

---

## Possible Answer

### Step-by-step ROP

1. Identify a gadget in the binary (e.g., “pop rdi; ret”) at a fixed address in the program or libc.
2. Compute the offset of system(), setuid(), exit(), and the string “/bin/sh” in the known libc.
3. Determine the runtime address of libc (e.g., look at maps in /proc fs or use a debuggers on a not-seuid copy of the binary)
4. Craft a payload:
  - a. Fill buf with 128 bytes, then overwrite saved RBP (8 bytes), then the return address.
  - b. Return address → address of pop rdi; ret gadget.
  - c. Next 8 bytes → 0 (64 bits)
  - d. Next → address of setuid() in libc
  - e. Next → address of pop rdi; ret gadget

- f. Next 8 bytes → address of the `"/bin/sh"` string in `libc`
- g. Next → address of `system()` in `libc`.
- h. [optional] Next → address of `exit` in `libc` (clean exit).

### Stack alignment

This is due to the x86-64 ABI for Linux (SysV): Some CPU instructions (e.g., `MOVAPS`) require the stack to be aligned to 16 bytes. These instructions are often called in `libc`.